

ST. XAVIER'S COLLEGE (AUTONOMOUS), MUMBAI

M.Sc BIG DATA ANALYTICS

Research Project Report

ATM CASH MANAGEMENT DECISION SUPPORT SYSTEM

Domain: **Banking**

Submitted by

Glen Noronha

Roll No.: 09

Under the guidance of

Mr. Ameya Chitnis

September 2026

Certificate of Originality

This is to certify that the project report entitled “AI-Powered ATM Cash Management Decision Support System” submitted in partial fulfilment of the requirements for the degree of M.Sc Big Data Analytics is a bona fide record of work carried out by Glen Noronha, Roll No. 09, under the supervision of Mr Ameya Chitnis .

The work presented in this report is original and has not been submitted, in part or in full, for the award of any other degree or diploma of this or any other institution.

Date: 03/10/2026

Place: Mumbai

Student Signature

Guide Signature

Acknowledgement

I would like to express my sincere gratitude to my project guide, Mr. Ameya Chitnis, for his valuable guidance, continuous support, and constructive feedback throughout the development of this research project.

I am also grateful to the faculty members of the M.Sc. Big Data Analytics programme at St. Xavier's College (Autonomous), Mumbai, for providing the academic guidance and learning environment necessary to undertake this project.

I would like to thank everyone who provided academic and technical support during the various stages of the project, including data analysis, exploratory analysis, feature engineering, modelling, evaluation, and documentation.

Finally, I acknowledge the support and encouragement received throughout the completion of this project, which contributed to the successful development of the AI-Powered ATM Cash Management Decision Support System.

Table of Contents

Certificate of Originality	ii
Acknowledgement.....	iii
Table of Contents.....	iv
List of Figures & Tables	vi
Abstract / Executive Summary.....	vii
Chapter 1: Introduction.....	1
1.1 Background & Motivation	1
1.2 Business Problem Statement	1
1.3 ML Problem Formulation	1
1.4 Objectives & Success Metrics	1
1.5 Scope & Constraints	2
Chapter 2: Data Understanding	3
2.1 Data Source(s).....	3
2.2 Data Dictionary Summary	3
2.3 Initial Data Audit	4
2.4 Target Variable Analysis.....	4
Chapter 3: Exploratory Data Analysis.....	5
3.1 Univariate Analysis.....	5
3.2 Bivariate Analysis	7
3.3 Multivariate Analysis.....	7
3.4 Key Insights Summary	8
Chapter 4: Data Cleaning.....	9
4.1 Missing Value Treatment	9
4.2 Outlier Treatment	9
4.3 Inconsistent & Impossible Values	9
4.4 Duplicate Handling	9
Chapter 5: Feature Engineering	10
5.1 Derived Features	10
5.2 Transformations.....	10
5.3 Encoding Strategy	10
5.4 Feature Engineering Summary Table	10
Chapter 6: Feature Selection	11
6.1 Method(s) Used	11
6.2 Final Feature List	11
Chapter 7: Data Preprocessing & Train/Test Strategy	12
7.1 Train/Test Split Strategy	12
7.2 Scaling & Encoding Pipeline	12
7.3 Handling Class Imbalance	12
7.4 Cross-Validation Design	12
Chapter 8: Model Development.....	13
8.1 Baseline Model.....	13
8.2 Candidate Models.....	13
8.3 Model Selection Rationale	14
Chapter 9: Hyperparameter Tuning	15
9.1 Search Method	15
9.2 Search Space	15
9.3 Best Parameters Found	15
Chapter 10: Model Evaluation & Validation	16
10.1 Classification Metrics	16
10.2 Discrimination Metrics.....	16
10.3 Decile / Lift / Gain Analysis.....	17
10.4 Calibration	17
10.5 Overfitting Diagnostics	18

10.6 Stability Analysis (PSI / CSI)	19
Chapter 11: Business Impact & Recommendations	20
11.1 Score-to-Action Mapping	20
11.2 Cost-Benefit Analysis	20
11.3 Deployment & Monitoring Recommendations	21
Chapter 12: Conclusion & Future Work	22
12.1 Summary of Findings	22
12.2 Limitations	22
12.3 Future Improvements	22
References	23
Appendix	24
A. Full Data Dictionary	24
B. Key Code Snippets	24
C. Additional Charts & Tables	25

List of Figures & Tables

List of Figures

- Figure 1: Daily withdrawal trend over the study period
- Figure 2: Day-of-week withdrawal pattern
- Figure 3: Salary-period withdrawal pattern
- Figure 4: Holiday withdrawal effect
- Figure 5: Monthly seasonality
- Figure 6: Outlier and skewness assessment
- Figure 7: Bivariate relationship with 3-day low-cash target
- Figure 8: Correlation heatmap
- Figure 9: Precision–Recall curve for the 3-day XGBoost classifier
- Figure 10: Feature importance for the 3-day classifier
- Figure 11: Top ATM replenishment priorities
- Figure 12: Decision-engine backtest
- Figure 13: Decile/lift/gain analysis
- Figure 14: Calibration curve
- Figure 15: Learning curve

List of Tables

- Table 1: Dataset and data-type summary
- Table 2: Target definition and class balance
- Table 3: Outlier and skewness assessment
- Table 4: Feature engineering summary
- Table 5: Final model feature set
- Table 6: Forecasting model comparison
- Table 7: Classification model comparison
- Table 8: Hyperparameter search space and selected parameters
- Table 9: Classification metrics at threshold 0.5
- Table 10: Decile/lift/gain results
- Table 11: Backtest results by daily visit capacity
- Table 12: Illustrative cost-benefit comparison
- Table 13: Full data dictionary

Abstract

ATM cash replenishment is a recurring operational decision in which insufficient cash can disrupt customer service while excessive replenishment can increase handling and logistics costs. This project develops an AI-powered decision-support system that ranks ATMs by near-term low-cash risk and converts the ranking into an actionable replenishment recommendation.

The study uses 222,356 transaction records covering 60 ATMs, 25 branches and 10 cities from January 2023 to December 2024. The transaction-level data were cleaned and transformed into an ATM-day panel of 43,797 observations. Exploratory analysis examined temporal, calendar, location and cash-balance patterns. Because the supplied cash-shortage flag contained only one positive record, a project-defined Low Cash Alert was created when end-of-day cash fell below 20% of ATM capacity. Leakage-safe future targets were then constructed for 1-day and 3-day horizons. A pooled XGBoost model was used for withdrawal-demand forecasting and a separate XGBoost classifier was developed for 3-day low-cash risk, with Logistic Regression and Random Forest used as comparison models and ARIMA used as a representative single-ATM benchmark.

The selected 3-day classifier achieved PR-AUC 0.527, ROC-AUC 0.917 and recall 0.796 on the held-out test period. Decile analysis showed that the highest-risk 10% of ATM-days contained about 70% of observed low-cash events. In the decision-engine backtest, model-driven prioritisation reduced low-cash days by 25%, 53% and 68.8% at daily visit capacities of 8, 12 and 16 respectively, although it performed 8% worse than the fixed schedule at a very tight capacity of 5 visits per day and required more trips.

The resulting system should therefore be viewed as a decision-support and prioritisation tool rather than an autonomous replenishment controller. Deployment should use a top-N risk ranking based on the institution's actual daily visit capacity, combined with a minimum-rotation rule and ongoing monitoring of model performance and population stability. The cost-benefit analysis remains sensitivity-dependent because the dataset does not provide a direct monetary cost for a missed low-cash event.

Chapter 1: Introduction

1.1 Background & Motivation

ATM networks must maintain sufficient cash at multiple locations whose withdrawal demand changes across days, weekends, salary periods, holidays and local operating conditions. A replenishment decision therefore has two competing objectives: reduce the risk that an ATM becomes low on cash while avoiding unnecessary visits and excessive cash held in machines.

The project focuses on converting historical transaction and cash-balance information into a daily decision-support process. Rather than producing only a numerical forecast, the system is designed to answer an operational question: which ATMs should be prioritised for replenishment, and how much cash should be considered for loading?

1.2 Business Problem Statement

The business problem is to support ATM cash-operations teams in deciding which ATMs require attention before their cash level becomes critically low. A fixed calendar-based replenishment schedule can visit ATMs that do not currently need cash while missing changes in demand or cash position. The proposed system uses historical withdrawal behaviour, current cash relative to capacity, calendar effects, ATM identity/location and replenishment history to rank ATMs by near-term low-cash risk and support a constrained daily visit plan.

1.3 ML Problem Formulation

The modelling unit is one ATM-day. The primary classification target is `'low_cash_next_3d'`, defined as whether the ATM will have a project-defined Low Cash Alert on any of the next three calendar days. The underlying Low Cash Alert is 1 when end-of-day cash is below 20% of the ATM's cash capacity. A secondary target, `'low_cash_next_1d'`, predicts the next-day alert. Prediction is made using information available at the current ATM-day only; same-day future cash outcomes are excluded to avoid target leakage. A separate regression component forecasts next-day withdrawal amount using past-only lag and rolling features.

1.4 Objectives & Success Metrics

Objectives:

1. Build a leakage-safe ATM-day analytical dataset.
2. Forecast short-term withdrawal demand and compare it with simple baselines and a classical ARIMA benchmark.
3. Predict future low-cash risk with an emphasis on recall and ranking quality under severe class imbalance.
4. Convert model scores into replenishment priorities and test whether the resulting policy improves the number of low-cash ATM-days under constrained visit capacity.

Project-defined success criteria were a reduction of at least 25% in low-cash ATM-days at a feasible daily visit capacity of eight or more visits, and a PR-AUC of at least 0.40 for the primary 3-day classifier. The final results exceeded the statistical target and met the service-side target for capacities of 8, 12 and 16 visits per day.

1.5 Scope & Constraints

The study covers the project-provided transaction dataset for January 2023 to December 2024 and focuses on daily ATM cash-management decisions. The analysis was performed with Python-based data science tools, including pandas, NumPy, scikit-learn, XGBoost, statsmodels and Matplotlib.

Important constraints are the absence of a directly observed shortage outcome suitable for supervised learning, the need to define a project-specific Low Cash Alert, moderate accuracy of exact withdrawal forecasting, and the absence of a directly observed monetary cost for a missed low-cash event. The decision engine is therefore a prototype for decision support rather than a production banking control system. The supplied project materials also do not document a live-bank data owner, access mechanism or regulatory approval process.

Chapter 2: Data Understanding

1.6 Literature Review & Research Gap

Prior research treats ATM cash management as a forecasting and replenishment problem rather than only a transaction-level classification problem. Venkatesh et al. studied clustering of ATMs with similar withdrawal patterns and then used neural-network models for grouped demand forecasting. Ekinci, Lu and Duman extended the problem to replenishment optimisation by linking group-demand forecasts to cash-loading decisions and resource use. More recent studies compare statistical and machine-learning methods, including ARIMA, Random Forest and neural networks, and report that model performance can vary with market conditions and ATM characteristics. Research on Indian-bank ATM demand has also examined calendar variables such as day-of-week and compared ARIMA, tree-based models and deep-learning approaches. These studies motivate a hybrid design in which forecasting, risk classification and operational decision rules are connected rather than treated as isolated modelling exercises.

The present project differs in emphasis by constructing a single decision-support workflow around a pooled multi-ATM model, a future low-cash risk target, ranked replenishment priorities, and an explicit policy backtest. The research gap addressed at project level is therefore the integration of leakage-safe modelling with an operational ranking policy and an out-of-sample decision-engine evaluation.

1.7 Research Design & Methodology

The research follows a structured analytical pipeline: (1) transaction-level data audit and quality assessment; (2) cleaning and conversion to an ATM-day decision grain; (3) exploratory analysis of demand, cash levels and temporal effects; (4) feature engineering using past-only lags, rolling demand, calendar indicators and replenishment history; (5) chronological train/test separation; (6) baseline, Logistic Regression, Random Forest, XGBoost and ARIMA comparisons; (7) hyperparameter search using time-aware validation; (8) evaluation using classification, ranking, calibration, overfitting and stability diagnostics; and (9) translation of model scores into replenishment actions followed by a policy backtest and sensitivity-based cost analysis.

2.1 Data Source(s)

The analysis uses a **synthetic** file ``atm_cash_management_dataset_fixed.csv``. The notebook records **222,356 transaction observations** covering **60 ATMs**, **25 branches** and **10 cities from 2023-01-01 to 2024-12-30**. The dataset contains *transaction*, *cash-balance*, *replenishment*, *location* and *weather-context* fields.

2.2 Data Dictionary Summary

The raw dataset contains **29 columns: 13 string fields, 8 floating-point fields, 7 integer fields and 1 datetime field**. Important variables include ``transaction_amount`` for withdrawal demand, ``cash_available_after`` for cash remaining after a transaction, ``atm_cash_capacity`` for normalising cash position, ``replenishment_flag`` and ``replenishment_cost`` for replenishment activity, and ``atm_status``, which was investigated for leakage before modelling.

The modelling dataset is an ATM-day panel containing 43,797 rows and 24 core engineered columns before the later addition of ``days_since_replenishment`` for classification.

Type	Count	Examples
------	-------	----------

String	13	transaction_id, atm_id, city, transaction_type, card_type
Float	8	transaction_amount, cash_available_after, replenishment_cost, weather_temp_c
Integer	7	is_weekend, is_holiday, failed_transaction_flag, atm_cash_capacity
Datetime	1	transaction_date

2.3 Initial Data Audit

The raw dataset contains **222,356 rows** and **29 columns** with **no missing values**. `transaction\_id` is unique, with **zero duplicate IDs**. Withdrawal cash-accounting checks found 561 rows (0.35% of 160,227 withdrawal rows) with a difference greater than ₹1 between the reported cash movement and the expected movement. This check was retained as a data-quality diagnostic rather than used as a blanket deletion rule.

The descriptive statistics show **transaction amounts from ₹0 to ₹50,000**, **mean withdrawal transaction value of approximately ₹3,609**, **mean end-of-transaction cash of approximately ₹86,658**, and **ATM capacities generally around ₹150,000** with some higher-capacity machines.

2.4 Target Variable Analysis

The supplied `cash\_shortage\_flag` is **unsuitable** as a supervised target: **only 1 of 222,356** transactions is positive (0.0004%). The project therefore defines a **Low Cash Alert at below 20% of ATM capacity**. This produces **505 low-cash ATM-days out of 43,797** (1.15%). After shifting the alert into future targets, `low\_cash\_next\_1d` has a **positive rate of about 1.15%** and `low\_cash\_next\_3d` **about 3.22%**.

This extreme **imbalance makes accuracy misleading**. A classifier that always predicts “no alert” would achieve approximately 97.4% accuracy on the 3-day target while detecting no real alerts. Precision–Recall AUC, recall, ranking/lift and related metrics are therefore emphasised.

Target / Label	Definition	Positive rate
cash_shortage_flag	Original supplied shortage indicator	1 / 222,356 = 0.0004%
low_cash_alert	End-of-day cash < 20% of capacity	505 / 43,797 = 1.15%
low_cash_next_1d	Low Cash Alert on next day	≈1.15%
low_cash_next_3d	Low Cash Alert on any next 3 days	≈3.22%

Chapter 3: Exploratory Data Analysis

3.1 Univariate Analysis

Withdrawal demand is strongly right-skewed: transaction-level withdrawal amount has skewness of approximately 2.16, while **replenishment cost is more strongly right-skewed** at approximately 3.35. IQR rules flag about 4.65% of withdrawal amounts and 5.48% of replenishment costs, but the flagged observations were investigated rather than automatically removed. **Withdrawal values are multiples of ₹100 and the ₹50,000 upper value is consistent with an ATM withdrawal limit in the supplied data. End-of-day cash is approximately symmetric at the ATM-day level.**

Calendar EDA also showed meaningful demand differences: weekend withdrawals were approximately 24% higher than weekday withdrawals, holiday daily withdrawal value was approximately 53% higher, and salary-period demand was elevated around the beginning of the month. **These patterns justify retaining calendar variables as candidate predictors.**

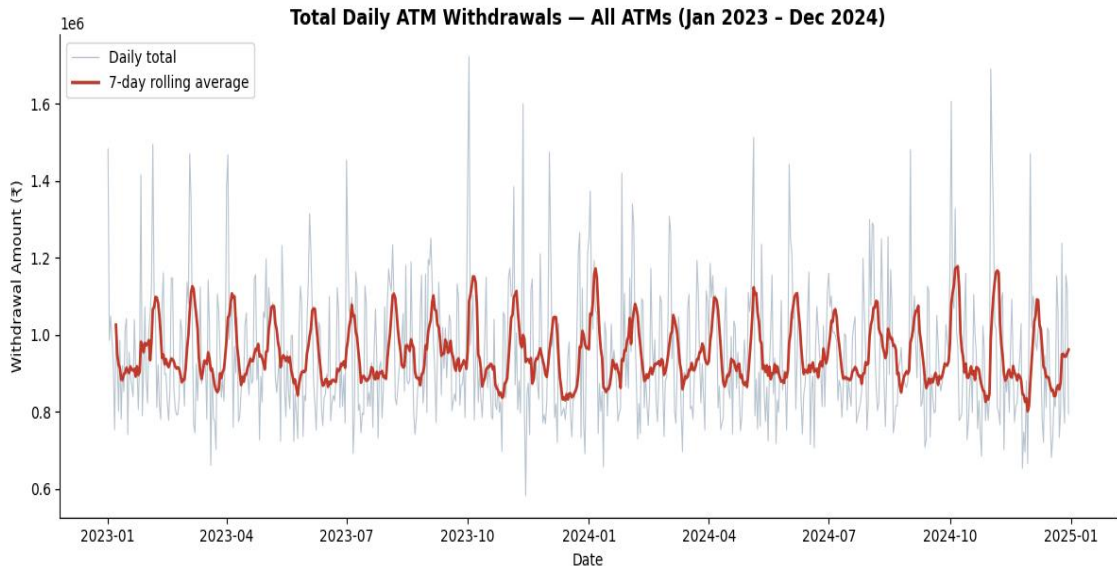


Figure 1: Daily withdrawal trend over the study period

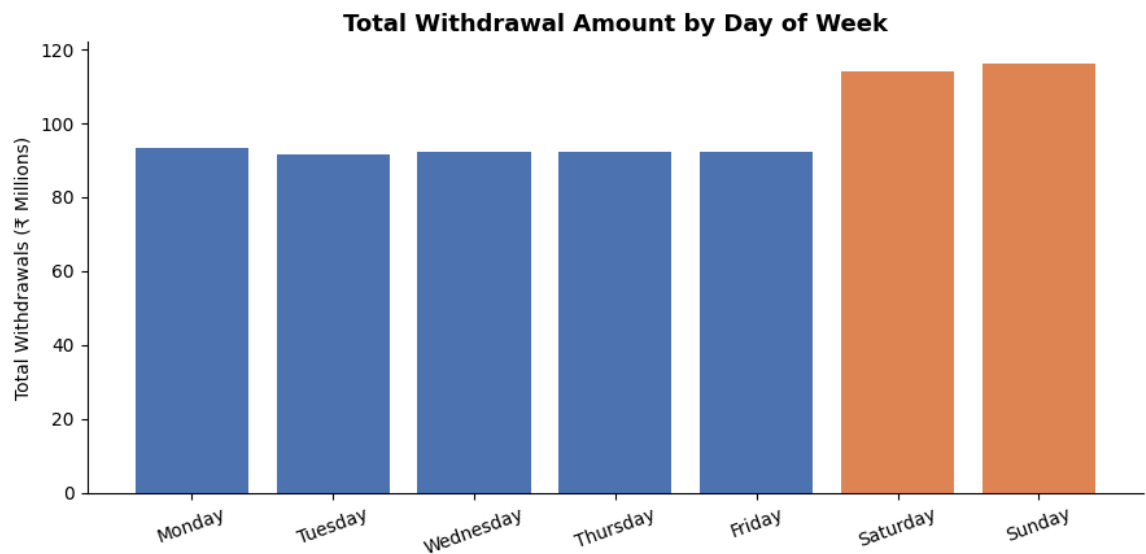


Figure 2: Day-of-week withdrawal pattern

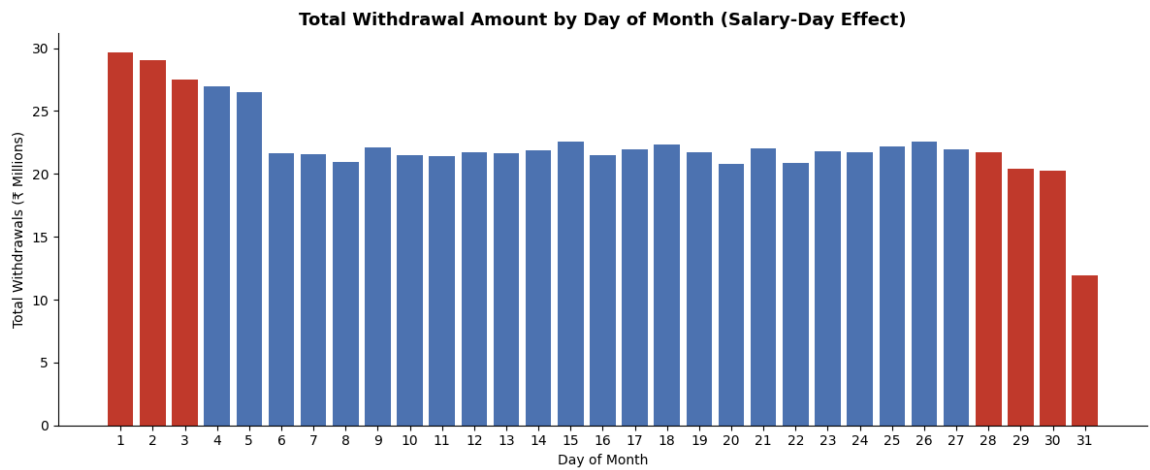


Figure 3: Salary-period withdrawal pattern

Holiday Effect: Two Different Views of the Same Data

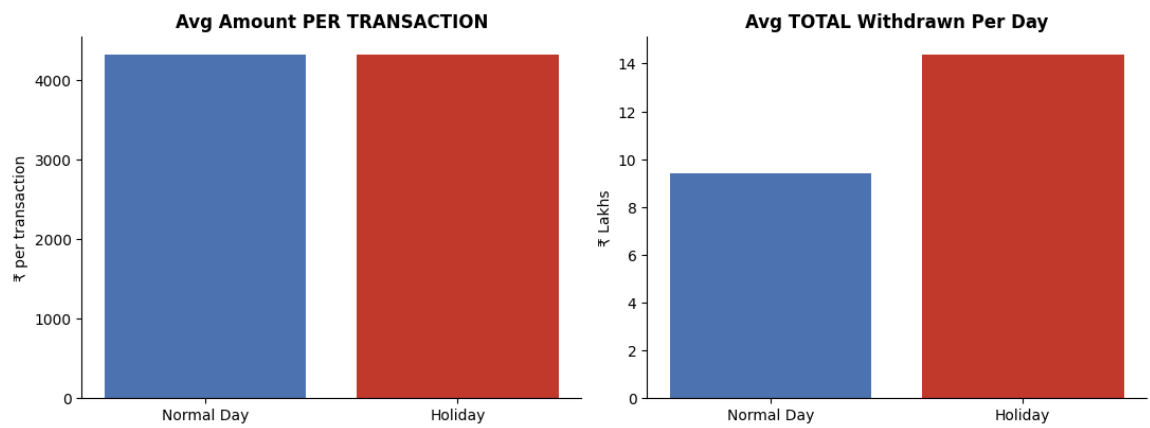


Figure 4: Holiday withdrawal effect

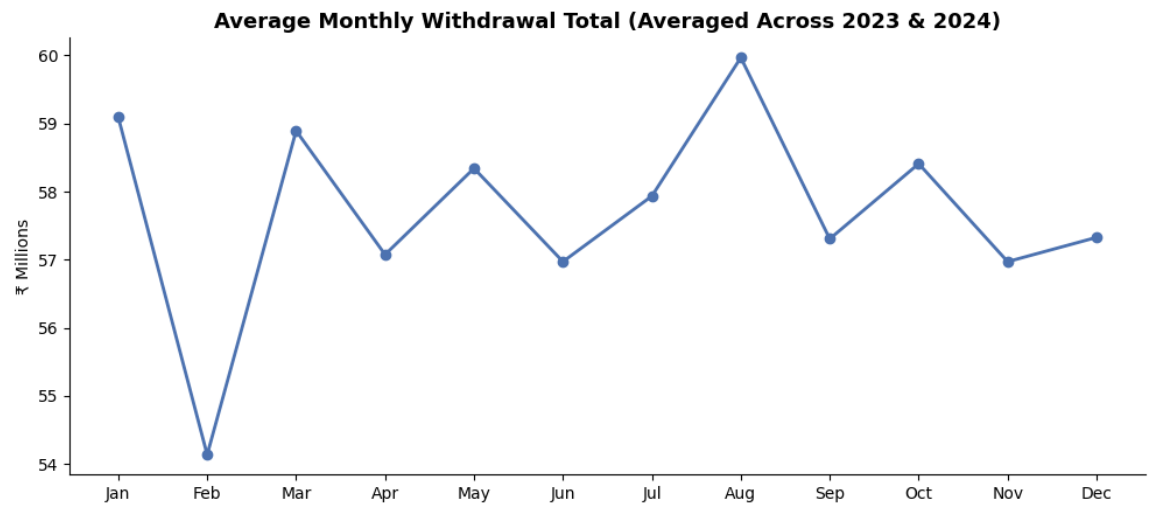


Figure 5: Monthly seasonality

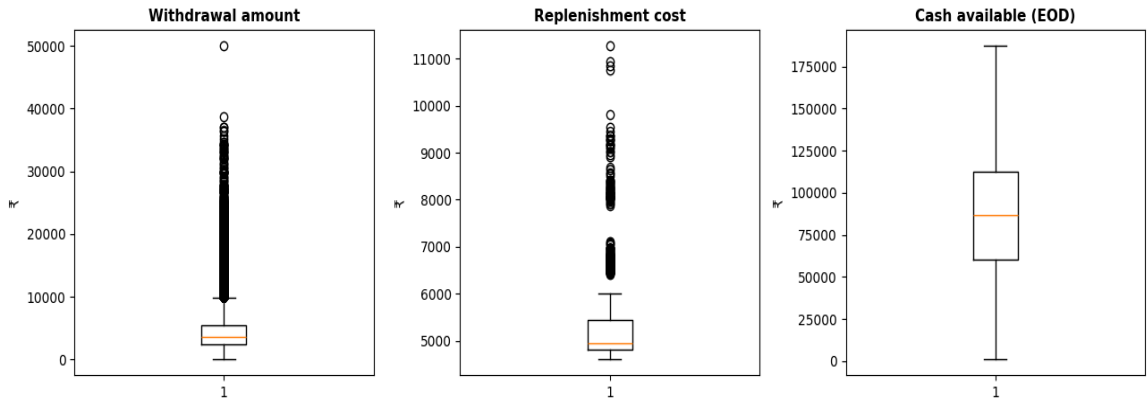


Figure 6: Outlier and skewness assessment for key monetary variables

Variable	Skew / IQR flag rate	Observed range / note
Transaction amount	Skew ≈ 2.16 ; $\approx 4.65\%$ flagged	₹0–₹50,000; non-zero values are ₹100 multiples
Replenishment cost	Skew ≈ 3.35 ; $\approx 5.48\%$ flagged	₹4,604–₹11,270; high tail linked to emergency type
Cash available after	Skew ≈ 0.04 ; 0% flagged	$\approx ₹896$ –₹187,299

3.2 Bivariate Analysis

The strongest direct relationship with the future low-cash target is the current cash position. ATM-days that subsequently enter a low-cash state within three days have visibly lower current cash levels than non-event days. Day-of-week differences in the target itself are weaker, even though day-of-week affects total withdrawal demand. This distinction is important: a feature can influence demand without being a strong standalone predictor of future low-cash status.

The bivariate findings therefore support `pct_capacity_eod` as a core risk feature while treating calendar variables as supporting context rather than relying on them alone.

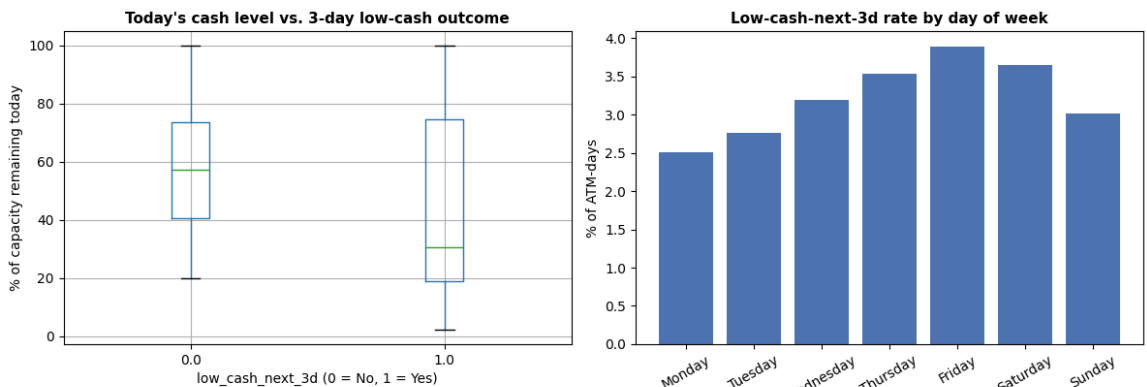


Figure 7: Bivariate relationship between current cash position and future low-cash risk

3.3 Multivariate Analysis

The correlation analysis shows **strong relationships among withdrawal amount and its own lag/rolling derivatives**. These correlations are expected because the variables deliberately encode recent withdrawal history. **No unexpected high correlation was found among the main independent predictors** such as current cash percentage, days since replenishment and calendar indicators. The final feature-selection process therefore focused more on leakage, operational

relevance and predictive contribution than on removing features solely because of correlation among intentionally related time-history variables.

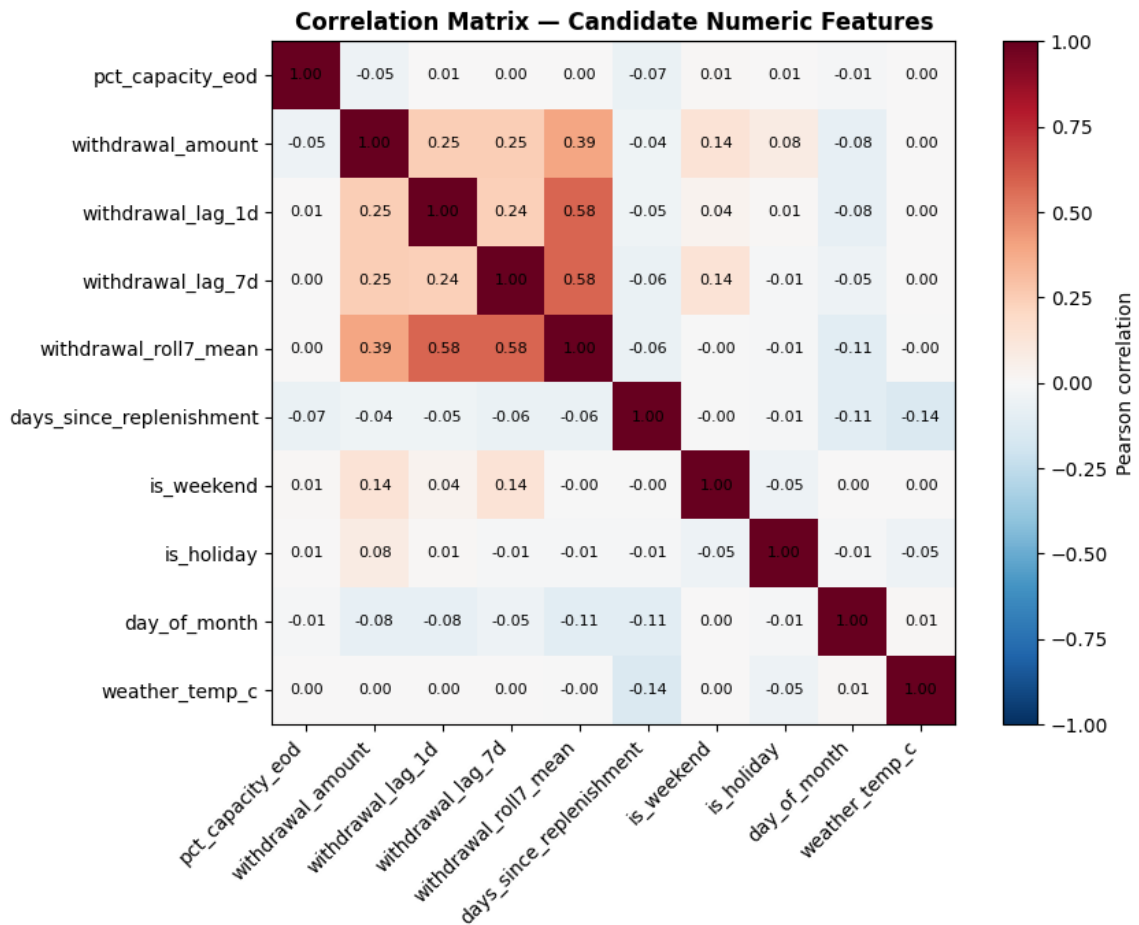


Figure 8: Correlation heatmap of engineered numeric predictors

3.4 Key Insights Summary

- The raw shortage label is unusable for modelling because it contains only one positive observation.
- A 20% capacity rule creates a workable 1.15% low-cash event rate at the ATM-day level.
- Withdrawal demand is highly skewed, but large withdrawals appear to be legitimate transactions rather than noise.
- Weekends and holidays show elevated withdrawal activity.
- Salary-period effects are visible around the beginning of the month.
- ATM identity explains a large share of withdrawal-demand variation; an identity-only model achieved $R^2 \approx 0.239$ versus ≈ 0.297 for the full XGBoost model.
- Exact daily withdrawal forecasting is only moderately predictable, limiting the precision of refill quantities.
- Current cash percentage and replenishment history are more operationally informative for low-cash risk than calendar variables alone.
- A ranking policy is more appropriate than a fixed probability threshold because daily replenishment capacity is a business constraint.
- Policy evaluation must be separated from classifier evaluation; a strong classifier does not automatically imply a good replenishment policy.

Chapter 4: Data Cleaning

4.1 Missing Value Treatment

No missing values were found in the raw 222,356-row transaction dataset. When constructing the complete ATM $\times$ date scaffold, days with no transactions were treated as genuine no-activity days after gap diagnostics supported that interpretation. Withdrawal and replenishment activity were zero-filled on such days, while cash balance was forward-filled within each ATM. Leading rows with no prior cash observation were dropped because their starting cash level could not be established.

4.2 Outlier Treatment

Outlier analysis was performed using skewness and IQR rules. The statistical flags were not automatically deleted because the extreme values were interpretable: withdrawal values were valid multiples of ₹100 and replenishment-cost tails were associated with emergency replenishment. The decision was therefore to retain genuine operational extremes and use them as signal rather than distort the data through blanket winsorisation or deletion.

4.3 Inconsistent & Impossible Values

Zero-value withdrawals were investigated separately. There were 1,256 such rows (0.78% of withdrawal rows); 1,252 were marked successful and none changed the cash balance. The smallest non-zero withdrawal was ₹500 and all non-zero withdrawals were exact multiples of ₹100. The 1,256 zero-value withdrawal rows were therefore removed as a data-generation artefact before aggregation.

The ``atm_status`` field was also audited. Its behaviour is closely tied to the cash-capacity threshold and was excluded from the primary low-cash classifier to avoid leakage.

4.4 Duplicate Handling

``transaction_id`` was checked for duplicates and the count was zero. No exact duplicate transaction rows were removed. The report therefore treats duplicate handling as an audit step rather than a cleaning operation.

Chapter 5: Feature Engineering

5.1 Derived Features

The transaction-level data were converted to an ATM-day decision grain. Derived features include end-of-day cash, total daily withdrawal amount, daily withdrawal count, replenishment activity, percentage of capacity remaining, 1-day and 7-day withdrawal lags, a 7-day rolling withdrawal mean, day-of-month, and days since the last replenishment. Future labels were generated using forward shifts so that the model predicts a future state rather than the current state.

5.2 Transformations

The raw monetary variables exhibited noticeable right-skewness, particularly transaction amounts and replenishment costs. However, the analysis indicated that the extreme values were not necessarily erroneous observations. Withdrawal amounts were valid multiples of ₹100, while higher replenishment costs were associated with emergency replenishment activity. Consequently, no blanket logarithmic transformation, winsorisation, or deletion was applied to the primary monetary variables.

Instead, the observed skewness was documented during exploratory analysis and the variables were retained in their operational form. This approach preserves potentially useful information about unusually high withdrawal activity and emergency replenishment events.

The main transformation was therefore focused on creating **comparable operational measures** rather than forcing the monetary distributions to become statistically symmetric. The most important normalised feature was:

```
pct_capacity_eod = (cash_eod / atm_cash_capacity) × 100
```

This transformation converts the absolute cash balance into a percentage of the ATM's available capacity. For example, ₹30,000 remaining in an ATM with a ₹100,000 capacity represents a substantially different operating condition from ₹30,000 remaining in an ATM with a ₹200,000 capacity. Expressing cash position as a percentage allows the model to compare these machines on a common scale.

Temporal transformations were also applied through lag and rolling features. Previous-day and previous-week withdrawal values were shifted so that only historical information was used. Similarly, the seven-day rolling withdrawal mean was constructed from past observations rather than future values.

These transformations were designed with **time-ordering and leakage prevention** in mind. Features that could reveal future cash conditions were excluded, and future observations were not allowed to influence the current prediction. The resulting feature set therefore represents information that could realistically be available at the time of the replenishment decision.

5.3 Encoding Strategy

Categorical variables used by the tree models were encoded numerically with **label encoding: ATM ID, location type and city. These encodings were fitted on the training data and then applied to the test data.** The final classification feature set contains 10 variables and deliberately excludes `atm\_status` because of leakage risk.

5.4 Feature Engineering Summary Table

The main engineered features were selected to represent four types of information: current cash position, recent demand, replenishment history and calendar/context effects. The resulting ATM-day dataset is designed around the actual operational decision rather than the original transaction grain.

Feature	Type	Source	Rationale
pct_capacity_eod	Continuous	cash_eod / capacity	Normalises current cash across ATM capacities
withdrawal_lag_1d	Continuous	previous ATM-day withdrawal	Captures very recent demand
withdrawal_lag_7d	Continuous	withdrawal 7 days earlier	Captures weekly recurrence
withdrawal_roll7_mean	Continuous	past 7 ATM-days	Smooths short-term demand
days_since_replenishment	Integer	replenishment history	Represents replenishment recency
day_of_month	Integer	transaction_date	Captures salary-period effects
is_weekend / is_holiday	Binary	calendar fields	Captures calendar demand shifts
ATM/location/city codes	Categorical codes	ATM metadata	Captures persistent ATM-level differences

Chapter 6: Feature Selection

6.1 Method(s) Used

Feature selection combined domain reasoning, leakage screening and embedded model information. Candidate variables were first assessed for whether they would be available at prediction time and whether they could directly encode the target. The final set was then evaluated through model performance and feature importance. This approach was preferred to a purely statistical filter because the project is a time-ordered operational decision problem and leakage prevention is critical.

6.2 Final Feature List

The final 3-day classifier uses: ``pct_capacity_eod``, ``withdrawal_lag_1d``, ``withdrawal_roll7_mean``, ``days_since_replenishment``, ``is_weekend``, ``is_holiday``, ``day_of_month``, ``atm_id_code``, ``loc_code``, and ``city_code``.

``atm_status`` was deliberately dropped because it is closely related to the low-cash condition. Same-day future cash measures were also excluded. The engineered lag and rolling variables were retained because they use only information available before the prediction date.

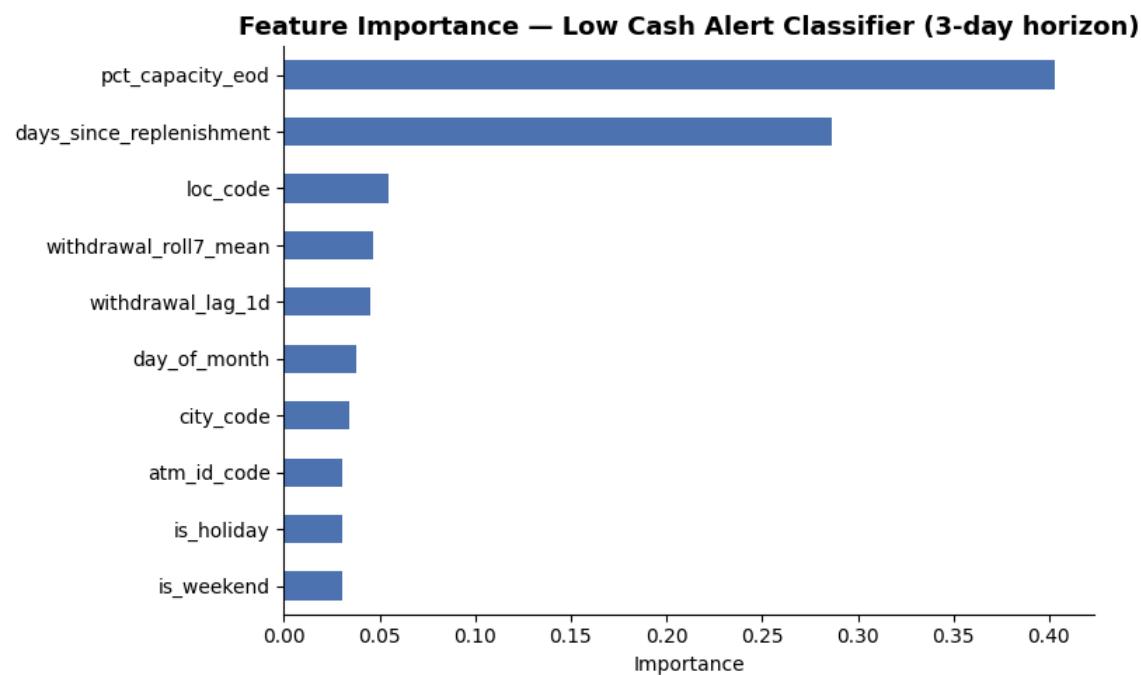


Figure 10: Feature importance for the 3-day low-cash classifier

Feature

Role

pct_capacity_eod	Current cash position; dominant operational signal
withdrawal_lag_1d	Recent demand
withdrawal_roll7_mean	Smoothed recent demand
days_since_replenishment	Time since last refill
is_weekend, is_holiday, day_of_month	Calendar context
atm_id_code, loc_code, city_code	ATM/network context
Excluded: atm_status	Leakage risk; closely tied to low-cash status

Chapter 7: Data Preprocessing & Train/Test Strategy

7.1 Train/Test Split Strategy

A chronological holdout was used rather than a random split. The **final 60 days of the project period were reserved as the out-of-sample test period**, giving **39,777 ATM-days for training and 3,600 ATM-days for testing** in the forecasting dataset; the classification dataset contains 3,540 complete test observations after lag/target exclusions. **All 60 ATMs appear in both periods, allowing the models to generalise across time while retaining ATM identity.**

7.2 Scaling & Encoding Pipeline

Logistic Regression used standardisation followed by class-weighted Logistic Regression. Tree-based models did not require feature scaling. Categorical codes were fitted using the training period and then transformed on the test period. The modelling workflow was kept chronological so that future observations were not used to fit preprocessing steps or features.

7.3 Handling Class Imbalance

The 3-day target has a test prevalence of about 2.6%, making class imbalance severe. **Logistic Regression used `class\_weight='balanced'`** (Giving more weight to imbalanced classes.) **XGBoost used `scale\_pos\_weight`** derived from the training class ratio. SMOTE was not used because synthetic oversampling can be problematic for time-ordered panel data and was not necessary for the **ranking**-based objective.

7.4 Cross-Validation Design

Hyperparameter tuning for the 3-day classifier used a chronological cross-validation approach with **four validation periods. In each period, the model was trained using earlier data and validated on later data. This approach was used to ensure that future information was not used when training the model.**

Chapter 8: Model Development

8.1 Baseline Model

Logistic Regression served as the interpretable baseline. For the 3-day target it achieved PR-AUC 0.074 and ROC-AUC 0.686, compared with PR-AUC 0.527 and ROC-AUC 0.917 for the selected XGBoost model. The baseline was useful because it established that the strong ranking performance was not simply the result of class imbalance.

8.2 Candidate Models

Candidate approaches included **XGBoost** and **Random Forest** for withdrawal forecasting; and **ARIMA** as a classical single-ATM forecasting benchmark.

For forecasting, XGBoost achieved MAE $\approx$ ₹7,793 and $R^2 \approx 0.297$ versus Random Forest MAE $\approx$ ₹7,908 and $R^2 \approx 0.277$. For 3-day classification, XGBoost achieved PR-AUC 0.527 and ROC-AUC 0.917 versus Random Forest PR-AUC 0.514 and ROC-AUC 0.914. Logistic Regression was substantially weaker on PR-AUC.

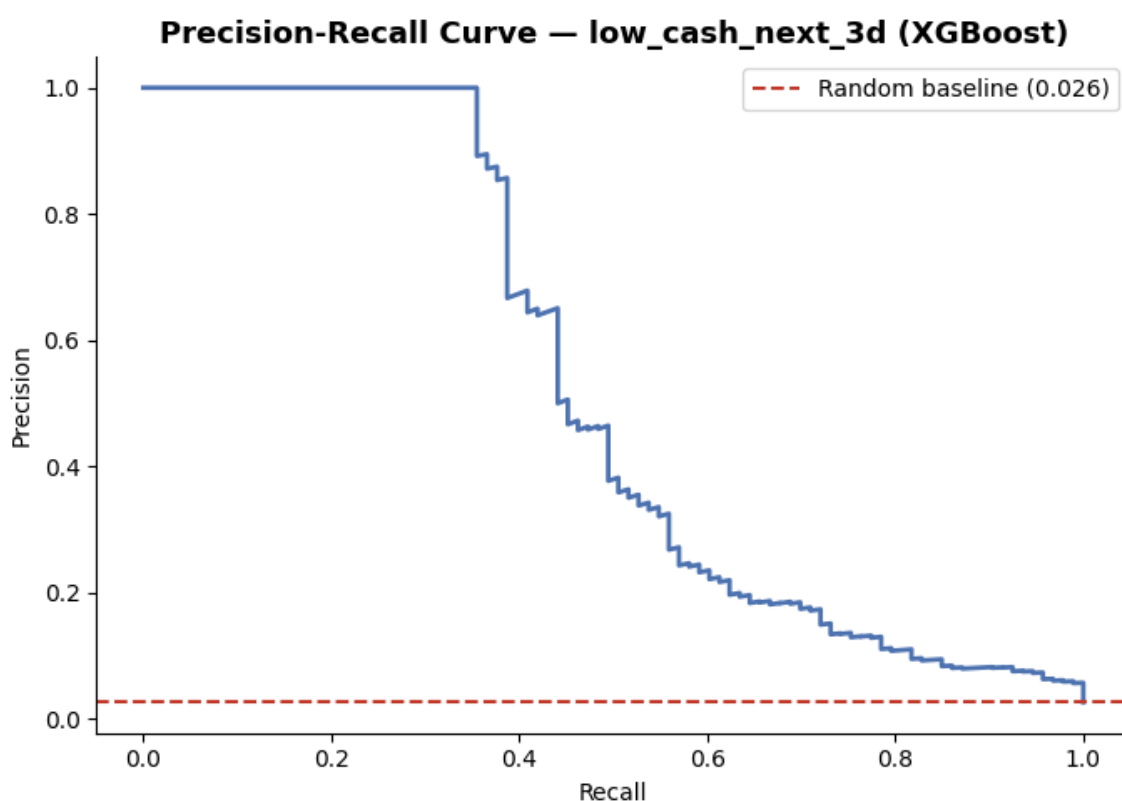


Figure 9: Precision–Recall curve for the 3-day XGBoost classifier

Task	Model	Key result	Decision
Forecast	7-day rolling mean	MAE $\approx$ ₹8,591	Baseline
Forecast	Random Forest	MAE $\approx$ ₹7,908; $R^2 \approx 0.277$	Candidate
Forecast	XGBoost pooled	MAE $\approx$ ₹7,793; $R^2 \approx 0.297$	Selected
Forecast	ARIMA(2,1,2), ATM0017	MAE $\approx$ ₹10,746	Representative benchmark
3-day classification	Logistic Regression	PR-AUC 0.074; ROC-AUC 0.686	Baseline
3-day classification	Random Forest	PR-AUC 0.514; ROC-AUC 0.914	Candidate
3-day classification	XGBoost	PR-AUC 0.527; ROC-AUC 0.917	Selected

8.3 Model Selection Rationale

XGBoost was carried forward because it provided the strongest held-out ranking performance among the tested tabular classifiers and slightly outperformed Random Forest on the forecasting task. It is also well suited to nonlinear interactions among cash position, recent demand, ATM identity and calendar variables.

ARIMA remains an important benchmark rather than a discarded method. On representative ATM0017, ARIMA(2,1,2) achieved MAE $\approx$ ₹10,746 versus pooled XGBoost MAE $\approx$ ₹10,772 on the same ATM. This indicates that classical time-series modelling can remain competitive for an individual series. Standard ARIMA is, however, a univariate model; a single global multi-ATM time-series model would require a different multi-series framework.

Chapter 9: Hyperparameter Tuning

9.1 Search Method

Randomized Search was used for the 3-day XGBoost classifier because the model has several interacting hyperparameters and a random search can explore a useful portion of the space without the computational cost of exhaustive enumeration. Validation was time-aware and restricted to the training period.

9.2 Search Space

The classifier search considered `n_estimators` in {100, 200, 300, 400, 500}; `max_depth` in {3, 4, 5, 6, 7}; `learning_rate` in {0.01, 0.03, 0.05, 0.08, 0.10}; `subsample` in {0.70, 0.85, 1.00}; `colsample_bytree` in {0.70, 0.85, 1.00}; and `min_child_weight` in {1, 3, 5}. Twenty random configurations were evaluated.

<i>Parameter</i>	<i>Search values</i>
<code>n_estimators</code>	100, 200, 300, 400, 500
<code>max_depth</code>	3, 4, 5, 6, 7
<code>learning_rate</code>	0.01, 0.03, 0.05, 0.08, 0.10
<code>subsample</code>	0.70, 0.85, 1.00
<code>colsample_bytree</code>	0.70, 0.85, 1.00
<code>min_child_weight</code>	1, 3, 5
Random configurations	20

9.3 Best Parameters Found

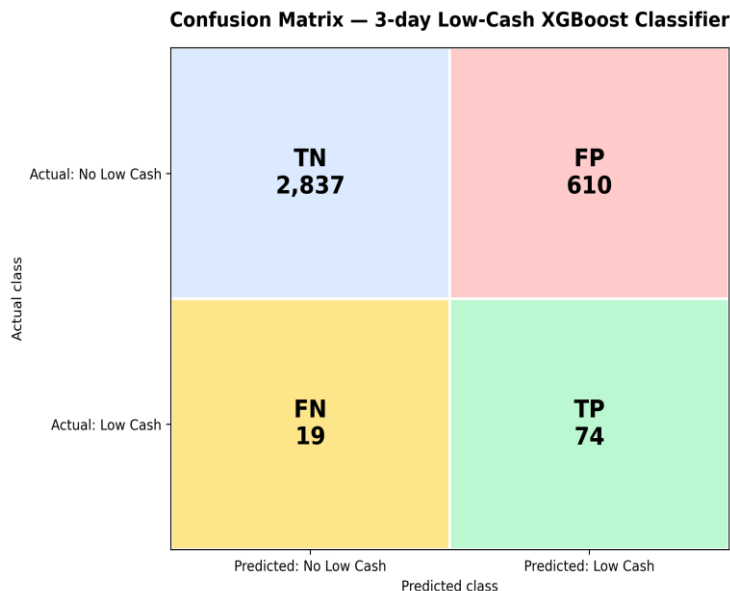
The best configuration found by time-aware cross-validation was `n_estimators=300`, `max_depth=3`, `learning_rate=0.05`, `subsample=0.85`, and `colsample_bytree=0.85`. On the held-out test set it achieved PR-AUC 0.527 and ROC-AUC 0.917

Configuration	<code>n_estimators</code>	<code>max_depth</code>	<code>learning_rate</code>	<code>subsample</code>	<code>colsample_bytree</code>	Test PR-AUC	Test ROC-AUC
Tuned CV winner	300	3	0.05	0.85	0.85	0.527	0.917

Chapter 10: Model Evaluation & Validation

10.1 Classification Metrics

At the diagnostic threshold of 0.5, the final 3-day classifier produced 74 true positives, 610 false positives, 19 false negatives and 2,837 true negatives. This corresponds to the reported precision of 0.108, recall of 0.796 and F1 of 0.190.



The 0.5 threshold is not the deployed decision rule. It is included because the report template requires a confusion matrix. **Operationally, the project uses a top-N ranking based on the available number of daily replenishment visits.**

Metric / diagnostic	Result
Precision	0.108
Recall	0.796
F1	0.190
True positives	74
False positives	610
False negatives	19
True negatives	2,837
Operational policy	Top-N ranking, not fixed 0.5 threshold

10.2 Discrimination Metrics

The final 3-day XGBoost classifier achieved ROC-AUC 0.917 and PR-AUC 0.527. The KS statistic was approximately 0.645 at a score threshold near 0.361, and the Gini coefficient was approximately 0.833 ($= 2 \times \text{ROC-AUC} - 1$). The PR-AUC is especially informative because the positive class is rare.

The relatively strong discrimination was checked against leakage: `atm\_status` was excluded, future cash values were not used as predictors, lag features were shifted before rolling, and the test period was strictly later than the training period.

10.3 Decile / Lift / Gain Analysis

The ranking analysis is one of the strongest results in the project. The highest-risk decile contained 65 of the 93 observed test-set events, capturing approximately 69.9% of all events with a lift of about 7.0× relative to the overall event rate. The top two deciles captured approximately 81.7% of events. This supports using the classifier as a prioritisation score rather than interpreting each raw score as a perfectly calibrated probability.

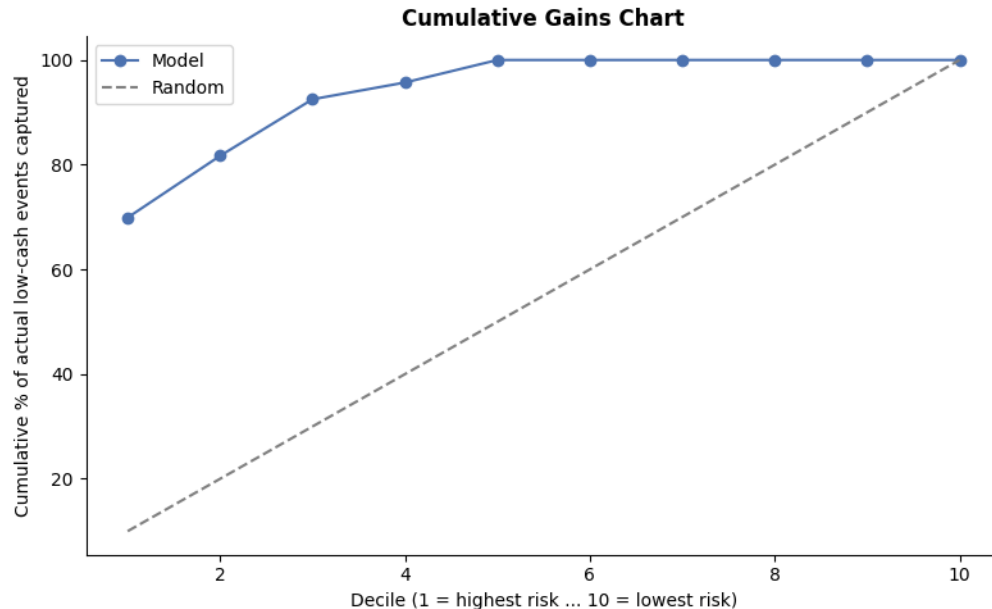


Figure 13: Cumulative gain / lift analysis

Decile (0 = highest risk)	Events	Event rate	Lift	Cumulative gain
0	65	18.4%	7.00×	69.9%
1	11	3.1%	1.18×	81.7%
2	10	2.8%	1.07×	92.5%
3	3	0.8%	0.30×	95.7%
4	4	1.1%	0.42×	100.0%
5–9	0 each	0%	0×	100.0%

10.4 Calibration

The calibration curve indicates that the raw XGBoost outputs are not reliable literal probabilities. For example, a region around a predicted score of approximately 0.73 corresponded to a much lower observed event rate. This behaviour is expected because `scale_pos_weight` changes the loss weighting to improve discrimination under class imbalance. The project therefore uses the model output primarily as a relative risk-ranking score.

A Brier score was not retained in the supplied notebook output, so no unsupported numerical Brier value is reported here. Probability calibration through Platt scaling or isotonic regression was not applied in the final pipeline.

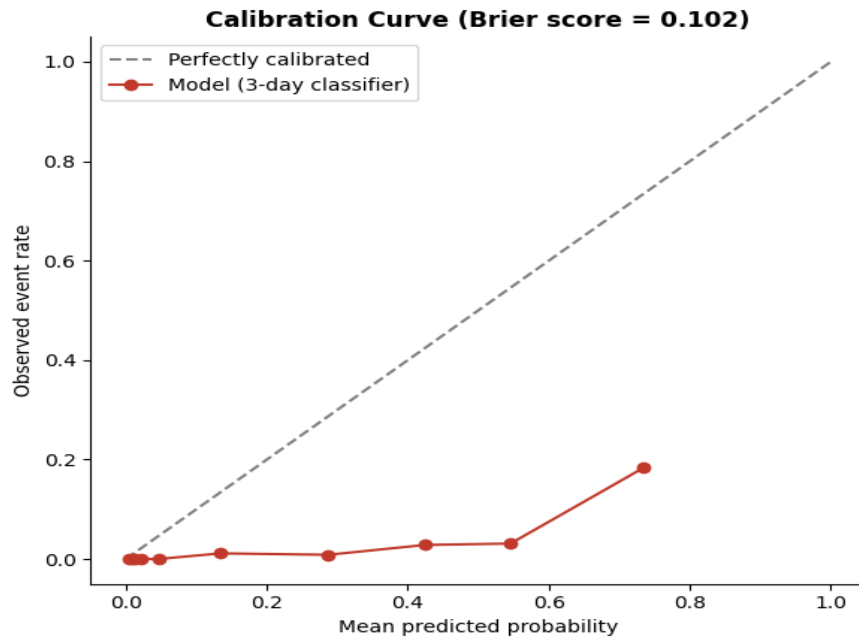


Figure 14: Calibration assessment of raw XGBoost scores

10.5 Overfitting Diagnostics

Train PR-AUC was approximately 0.633 versus 0.527 on the test set, a gap of 0.106. Train ROC-AUC was 0.953 versus 0.917 on test, a gap of 0.036. The learning curve showed test performance becoming broadly flat after roughly 60% of the training data.

The diagnosis is mild overfitting rather than catastrophic variance. The larger PR-AUC gap reflects the sensitivity of rare-event ranking to the positive class. The learning curve also suggests a bias/feature-information ceiling: simply adding more observations may not produce large gains unless richer demand or operational variables become available.

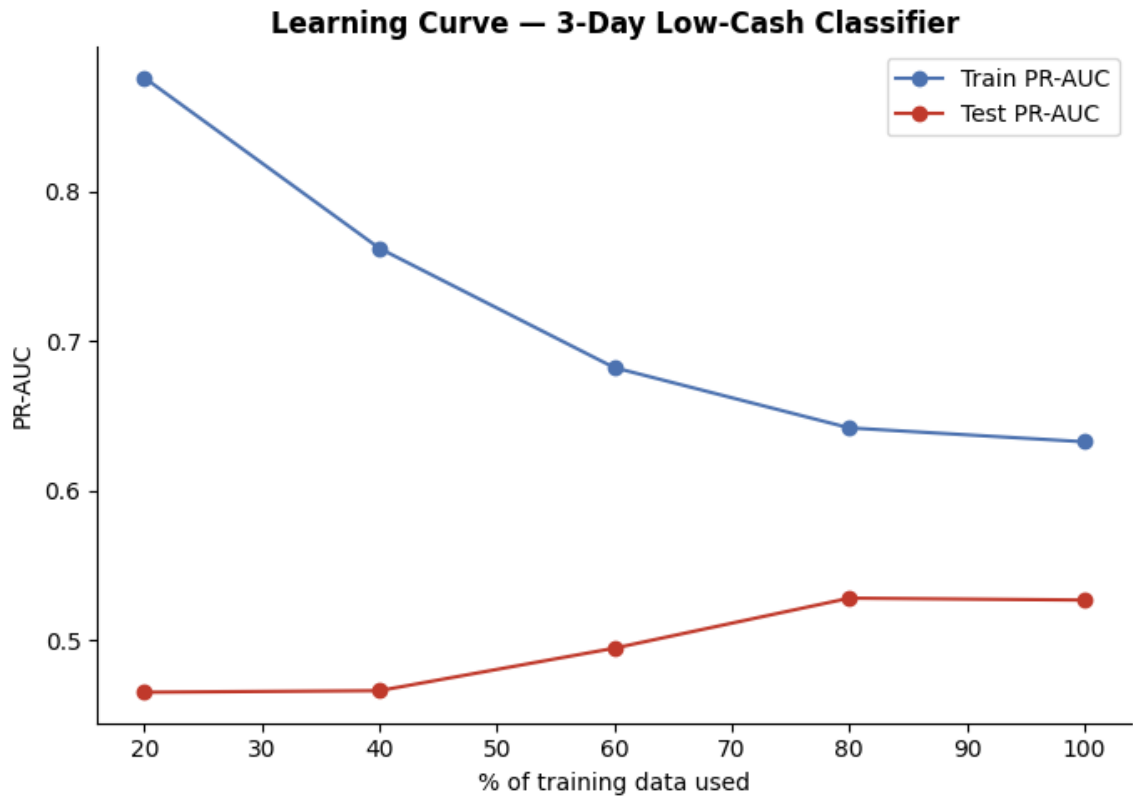


Figure 15: Learning-curve diagnostic

10.6 Stability Analysis (PSI / CSI)

The train-versus-test Population Stability Index for the model score was approximately 0.0023. Characteristic Stability Index values were approximately 0.0017 for `pct_capacity_eod` and 0.0009 for `days_since_replenishment`. These values are well below the commonly used 0.10 monitoring threshold and indicate stable distributions between the training period and the 60-day holdout.

This supports continued monitoring rather than immediate retraining. PSI should nevertheless be checked periodically after deployment because future changes in withdrawal behaviour, ATM network composition or operating policy could invalidate the training distribution.

Chapter 11: Business Impact & Recommendations

11.1 Score-to-Action Mapping

The decision engine treats the 3-day XGBoost score as a priority ranking. The highest-scoring ATMs are considered first, subject to the institution's daily visit capacity. A separate next-day risk score can flag urgent cases. The replenishment amount is calculated by a transparent rule: target cash is 90% of ATM capacity, so recommended refill = $\max(0, 0.90 \times \text{capacity} - \text{current cash})$. This rule is deliberately separated from the ML classifier so that the model predicts risk while the business rule determines a practical load quantity.

The backtest showed that ranking should be combined with a minimum-rotation policy in very tight-capacity settings, because pure top-risk selection can repeatedly prioritise the same ATMs and starve lower-risk machines.

Backtest results show a capacity-dependent trade-off:

Capacity	Fixed low-cash days	Model-driven low-cash days	Reduction	Fixed trips	Model trips
N=5	1,712	1,849	−8.0%	238	290
N=8	1,636	1,227	+25.0%	244	464
N=12	1,621	762	+53.0%	244	696
N=16	1,621	506	+68.8%	244	928

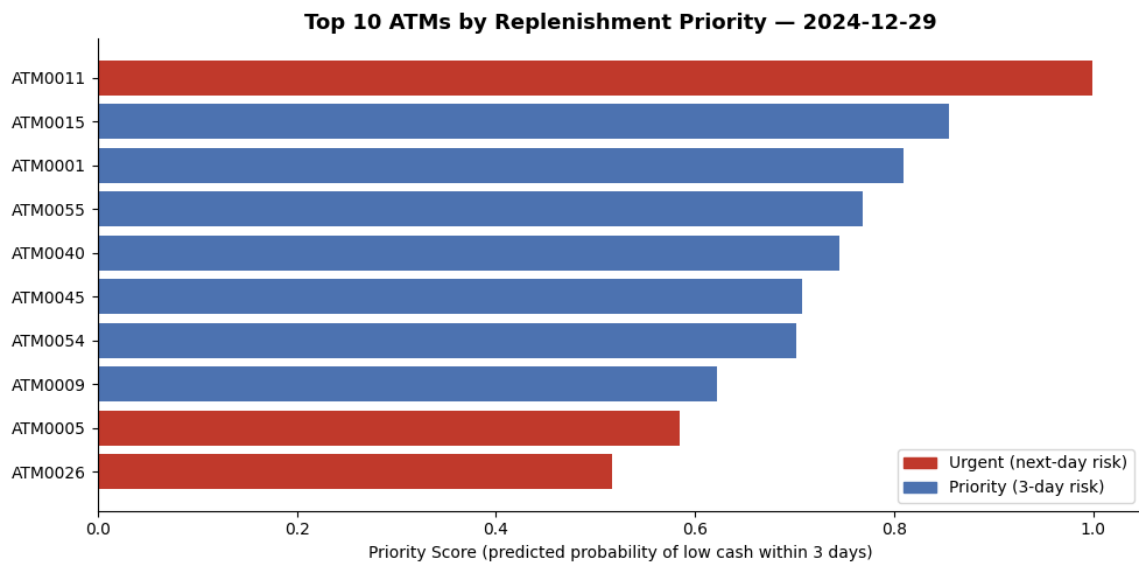


Figure 11: Top ATM replenishment priorities for the latest project snapshot

11.2 Cost-Benefit Analysis

The dataset-derived average scheduled replenishment trip cost is approximately ₹5,132, while the average emergency replenishment trip cost is approximately ₹8,597. The simulation policies act proactively, so the simplified cost model uses the scheduled rate for simulated trips.

Because the dataset does not contain a direct cost for a missed low-cash event, the analysis uses an explicitly illustrative assumption of ₹2,000 per low-cash ATM-day. Under this assumption, the model-driven policy requires more trips than the fixed schedule and has a higher simplified total cost at every tested capacity. Therefore the project does not claim unconditional monetary savings. Sensitivity analysis shows that at N=12 the model-driven policy becomes cheaper once the

assumed cost of a missed low-cash day is above roughly ₹2,700. The financial case should therefore be recalculated using the institution’s actual service-loss and logistics-cost estimates before deployment.

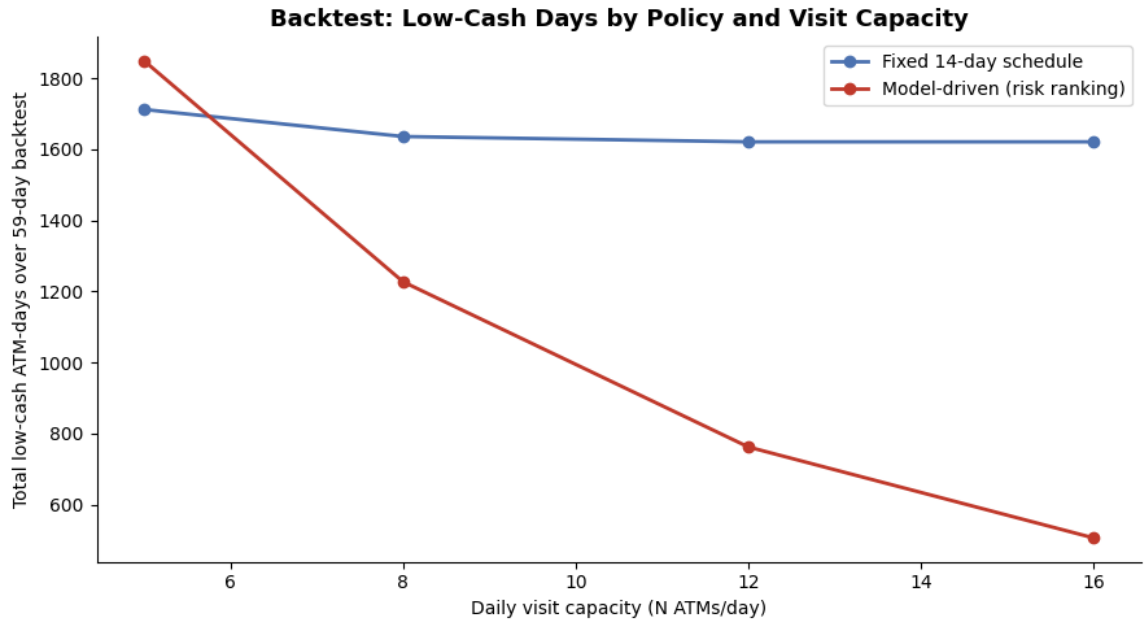


Figure 12: Decision-engine backtest across daily visit capacities

Daily capacity	Fixed low-cash days	Model low-cash days	Fixed trips	Model trips	Low-cash reduction	Simplified cost difference
5	1,712	1,849	238	290	−8.0%	+₹540,864
8	1,636	1,227	244	464	+25.0%	+₹311,040
12	1,621	762	244	696	+53.0%	+₹601,664
16	1,621	506	244	928	+68.8%	+₹1,280,288

11.3 Deployment & Monitoring Recommendations

Deployment should use a top-N ranking policy rather than a universal probability cutoff, with N equal to the actual daily replenishment capacity. In a production setting, the ranking should be combined with minimum-rotation constraints and an urgent override for imminent risk.

Monitoring should include monthly PSI on the score, CSI on major input features, PR-AUC/recall on newly labelled outcomes when labels become available, and the rate of low-cash incidents. A practical trigger is to investigate $PSI \geq 0.10$ and consider retraining or model review if PSI exceeds 0.25, alongside deterioration in PR-AUC/recall. Any new policy should also monitor trip count, emergency trips, low-cash days and idle cash to ensure that model performance translates into operational value.

Chapter 12: Conclusion & Future Work

12.1 Summary of Findings

This project developed an end-to-end ATM cash-management decision-support workflow from transaction data through cleaning, ATM-day aggregation, forecasting, low-cash classification, ranking, replenishment rules and policy backtesting. The pooled XGBoost classifier achieved PR-AUC 0.527 and ROC-AUC 0.917 on the held-out period, with the top risk decile capturing about 70% of observed low-cash events. The policy backtest demonstrated substantial service-side improvements at daily capacities of 8–16 visits, while also revealing a negative case at $N=5$ and a clear trade-off between fewer low-cash days and more replenishment trips. The result is therefore a practical prototype for prioritised cash operations, not a claim of guaranteed cost savings.

12.2 Limitations

- The original shortage label was too sparse to serve as a supervised target, so the project-defined 20% Low Cash Alert is a proxy rather than an observed bank shortage event.
- Exact withdrawal forecasting remains only moderately accurate ($R^2 \approx 0.30$), so refill quantities should not be interpreted as high-precision demand forecasts.
- ARIMA was evaluated only on representative ATM0017, not across all 60 ATMs.
- The refill target of 90% of capacity is a transparent prototype rule, not a demand-optimised inventory policy.
- The backtest assumes withdrawal demand is exogenous; it does not model transaction rejection when an ATM would actually run out of cash.
- The cost-benefit analysis requires an assumed missed-event cost because the dataset does not contain one.
- The project materials do not document live-bank deployment, governance, regulatory approval or a production data-refresh mechanism.

12.3 Future Improvements

Future work should incorporate actual transaction rejection/stockout outcomes, cash-in-transit constraints, ATM-specific replenishment lead times, denomination-level cash requirements, route optimisation, and a demand-aware refill target. A richer forecasting framework could compare global multi-series time-series models with gradient boosting and deep-learning approaches. Probability calibration could be added if the bank needs literal risk probabilities rather than ranking scores. Finally, a live pilot should measure service level, trip kilometres, emergency visits, idle cash and operational cost before and after deployment.

References

- [1] Venkatesh, K., Ravi, V., Prinzie, A., & Van den Poel, D. (2014). Cash demand forecasting in ATMs by clustering and neural networks. *European Journal of Operational Research*, 232(2), 383–392. <https://doi.org/10.1016/j.ejor.2013.07.027>
- [2] Ekinci, Y., Lu, J.-C., & Duman, E. (2015). Optimization of ATM cash replenishment with group-demand forecasts. *Expert Systems with Applications*, 42(7), 3480–3490. <https://doi.org/10.1016/j.eswa.2014.12.011>
- [3] Vangala, S., Ravi, V., & Vivek, Y. (2023). ATM cash demand forecasting in an Indian bank with chaos and hybrid deep learning networks. *Expert Systems with Applications*, 211, 118645. <https://doi.org/10.1016/j.eswa.2022.118645>
- [4] Suder, M., Gurgul, H., Barbosa, B., Machno, A., & Lach, Ł. (2024). Effectiveness of ATM withdrawal forecasting methods under different market conditions. *Technological Forecasting and Social Change*, 200, 123089. <https://doi.org/10.1016/j.techfore.2023.123089>
- [5] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- [6] Project-provided dataset: `atm\_cash\_management\_dataset\_fixed.csv`, covering January 2023–December 2024.
- [7] Project analysis notebook: `ATM\_Project.ipynb`, supplied for the MSc Big Data Analytics research project.

Appendix

A. Full Data Dictionary

Variable	Type	Role / Description
transaction_id	String	Unique transaction identifier
atm_id	String	ATM identifier
branch_id	String	Branch identifier
atm_location_type	String	Rural / semi-urban / urban location type
city	String	ATM city
latitude	Float	ATM latitude
longitude	Float	ATM longitude
transaction_date	Datetime	Transaction date
transaction_time	String	Transaction time
day_of_week	String	Day name
is_weekend	Integer	Weekend indicator
is_holiday	Integer	Holiday indicator
transaction_type	String	Transaction category
transaction_amount	Float	Transaction / withdrawal amount
card_type	String	Card category
transaction_status	String	Transaction status
failed_transaction_flag	Integer	Failure indicator
cash_available_before	Float	Cash before transaction
cash_available_after	Float	Cash after transaction
atm_cash_capacity	Integer	ATM cash capacity
cash_shortage_flag	Integer	Supplied shortage flag; unsuitable as ML target
replenishment_flag	Integer	Whether replenishment occurred
replenishment_amount	Float	Amount replenished
replenishment_type	String	Scheduled / Emergency
replenishment_cost	Float	Replenishment trip cost
days_since_last_replenishment	Integer	Source-level replenishment recency field
atm_status	String	ATM status; excluded from primary classifier due leakage risk
weather_temp_c	Float	Weather temperature
weather_condition	String	Weather condition

B. Key Code Snippets

```
Low Cash Alert:
panel['pct_capacity_eod'] = panel['cash_eod'] / panel['atm_cash_capacity'] * 100
panel['low_cash_alert'] = (panel['pct_capacity_eod'] < 20).astype(int)
Leakage-safe 3-day target:
panel['low_cash_next_3d'] = panel.groupby('atm_id')['low_cash_alert'].transform(
    lambda s: s.shift(-1).rolling(3, min_periods=1).max()
)
Past-only rolling feature:
panel['withdrawal_roll7_mean'] = panel.groupby('atm_id')['withdrawal_amount'].transform(
    lambda s: s.shift(1).rolling(7, min_periods=1).mean()
)
Final XGBoost classifier:
xgb.XGBClassifier(n_estimators=300, max_depth=4, learning_rate=0.05,
                  scale_pos_weight=pos_weight, random_state=42,
                  eval_metric='logloss')

Refill rule:
recommended_refill = max(0, 0.90 * atm_cash_capacity - current_cash)

panel = (
    df.groupby(['atm_id', 'transaction_date'])
      .agg(
          withdrawal_amount=('withdrawal_amount', 'sum'),
          withdrawal_count=('withdrawal_amount', 'count'),
          cash_eod=('cash_available_after', 'last'),
          atm_cash_capacity=('atm_cash_capacity', 'last'),
          is_weekend=('is_weekend', 'max'),
          is_holiday=('is_holiday', 'max')
      )
      .reset_index()
)

features = [
```

```

    'cash_eod',
    'pct_capacity_eod',
    'withdrawal_amount',
    'withdrawal_lag1',
    'withdrawal_lag7',
    'withdrawal_rol17_mean',
    'is_weekend',
    'is_holiday',
    'day_of_month',
    'days_since_replenishment'
]

# Leakage-prone fields such as atm_status
# and future cash outcomes are excluded.

random_search = RandomizedSearchCV(
    estimator=xgb_model,
    param_distributions=param_grid,
    n_iter=12,
    scoring='average_precision',
    cv=time_cv,
    random_state=42,
    n_jobs=-1
)

random_search.fit(X_train, y_train)

```

C. Additional Charts & Tables

The notebook contains additional exploratory and diagnostic visuals, including the overall withdrawal trend, day-of-week and salary-day effects, holiday effects, monthly seasonality, ARIMA versus XGBoost forecasting checks, feature-importance plots, calibration diagnostics and policy backtests.

